

Desafio Técnico: DevOps Engineer - Engenheiro(a) de Plataforma

1. Contexto

Nosso time de Platform Engineering mantém a plataforma interna sobre a qual os times de produto entregam software. A ideia deste desafio é conversarmos sobre engenharia de plataforma a partir de seu código.

2. O desafio

Colocar em funcionamento, em um cluster Kubernetes, a aplicação indicada na seção 5.

O trabalho:

- 1. **Provisionar.** A infraestrutura e as dependências necessárias para a aplicação funcionar são provisionadas de forma automatizada.
- 2. **Deployar.** O deployment da aplicação também é automatizado.
- 3. **Sustentar.** A aplicação precisa ser escalável e resiliente.

Ao final, você apresenta o resultado para nós: mostra ao vivo o que construiu e conta o caminho que escolheu. O ambiente pode já estar no ar quando a conversa começar.

3. Formato

Item	Definição
Prazo	7 dias corridos a partir do envio deste documento
Entrega	Um ou mais repositórios Git
Apresentação	60 minutos, você conduzindo, compartilhando tela e código
Uso de IA	Liberado e sem estigma. O que esperamos é que você consiga explicar as decisões do código

Se o prazo não couber bem na sua semana, é só nos avisar que ajustamos a data.

4. Ambiente

Onde rodar fica a seu critério, em cloud na sua conta ou localmente, desde que seja um ambiente Kubernetes. O ambiente fica sob seu controle. Não vamos provisionar nem rodar o projeto do nosso lado.

5. A aplicação

A aplicação está em <https://github.com/andreffcastro/todolist-app>. O repositório traz o que é necessário para colocá-la em funcionamento, e ela roda localmente.

Não há necessidade de escrever nem melhorar código de aplicação, mas se quiser mexer, fique à vontade.

6. Requisitos

Esperamos que o resultado siga as boas práticas de mercado.

Ficam a seu critério: quais ferramentas usar e quantas, como provisionar, como empacotar, como operar, e até onde levar o trabalho. Optamos por não indicar caminho nem ferramenta em nenhum ponto deste documento, porque o seu conhecimento e as suas decisões técnicas são parte do que queremos entender.

R1. Provisionamento automatizado

- Cluster Kubernetes criado por código.
- A infraestrutura e as dependências necessárias para a aplicação funcionar também são provisionadas de forma automatizada.
- O processo é repetível, sem passos manuais no console.

R2. Deployment automatizado

- Levar uma versão nova da aplicação ao cluster é automatizado.

R3. Acesso à aplicação

- A aplicação é acessível de fora do cluster, pelo navegador.

R4. Escalabilidade e resiliência

- A aplicação é escalável e resiliente.

R5. Documentação

- `README.md` que permita a outras pessoas entender e operar o que você construiu.
- Registro das decisões relevantes: o que você escolheu, por quê, e o que descartou.

7. Escopo

Os requisitos da seção 6 são o que pedimos. Ir além deles é opcional, e não ir também é uma decisão válida. Nossa sugestão é priorizar os requisitos antes de ampliar o escopo.

Se em algum momento você concluir que um requisito daqui está mal desenhado, fique à vontade para registrar o argumento e seguir como achar melhor.

8. Entregáveis

1. Repositórios com o histórico de commits.
2. `README.md` na raiz com o caminho de execução e um índice do que está onde.
3. Registro de decisões e o relato dos desafios que você encontrou pelo caminho.
4. Evidências de execução: logs, screenshots, etc.

Boa sorte, e qualquer coisa estamos por aqui.